

# PROTOCOLLO MQTT

# 1. ¿Qué es MQTT ?.

MQTT son las siglas MQ Telemetry Transport, aunque en primer lugar fue conocido como Message Queing Telemetry Transport.

Es un **protocolo de comunicación M2M (machine-to-machine) de tipo message queue**.

Está **basado en la pila TCP/IP** como base para la comunicación. En el caso de MQTT cada conexión se mantiene abierta y se "reutiliza" en cada comunicación. Es una diferencia, por ejemplo, a una petición HTTP 1.0 donde cada transmisión se realiza a través de conexión.

MQTT fue creado por el Dr. Andy Stanford-Clark de IBM y Arlen Nipper de Arcom (ahora Eurotech) en 1999 como un mecanismo para conectar dispositivos empleados en la industria petrolera.

Aunque inicialmente era un formato propietario, en 2010 fue liberado y pasó a ser un estándar en 2014 según la OASIS (Organization for the Advancement of Structured Information Standards).

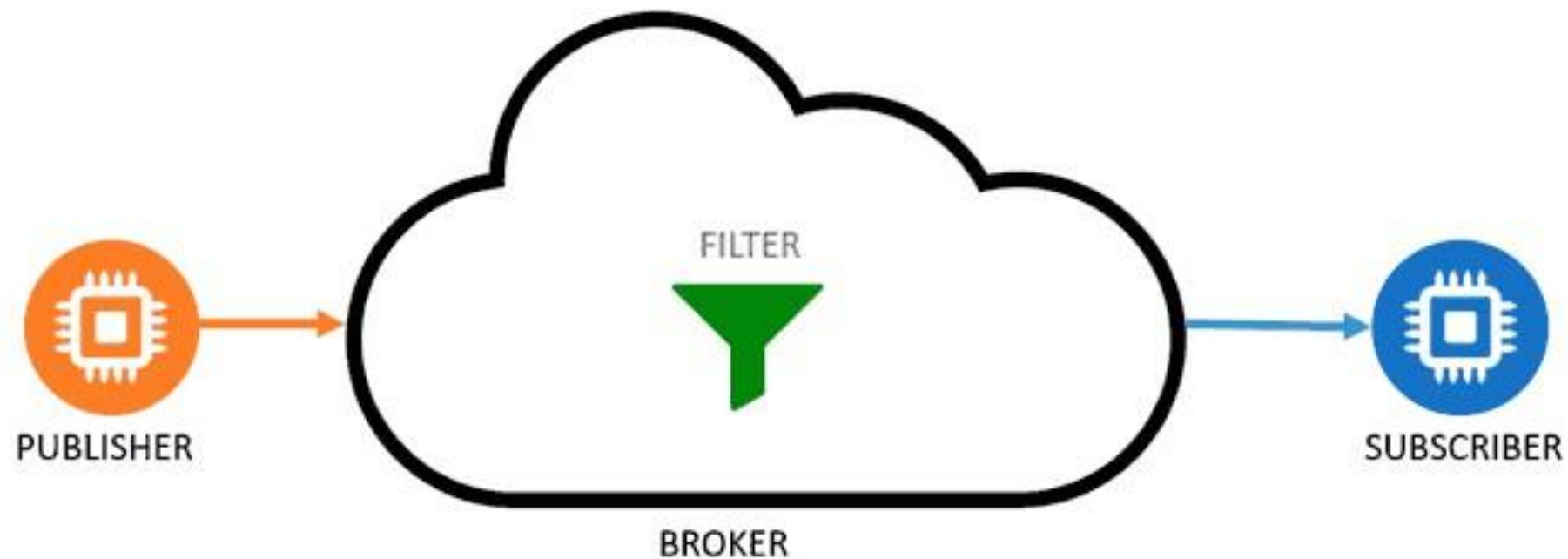
## 2. ¿Cómo funciona MQTT?

El funcionamiento del MQTT es **un servicio de mensajería push con patrón publicador/suscriptor (pub-sub)**, donde los clientes se conectan con un servidor central denominado broker.

Para filtrar los mensajes que son enviados a cada cliente **los mensajes se disponen en topics organizados jerárquicamente**.

Un cliente puede publicar un mensaje en un determinado topic.

Otros clientes pueden suscribirse a este topic, y el broker le hará llegar los mensajes suscritos.

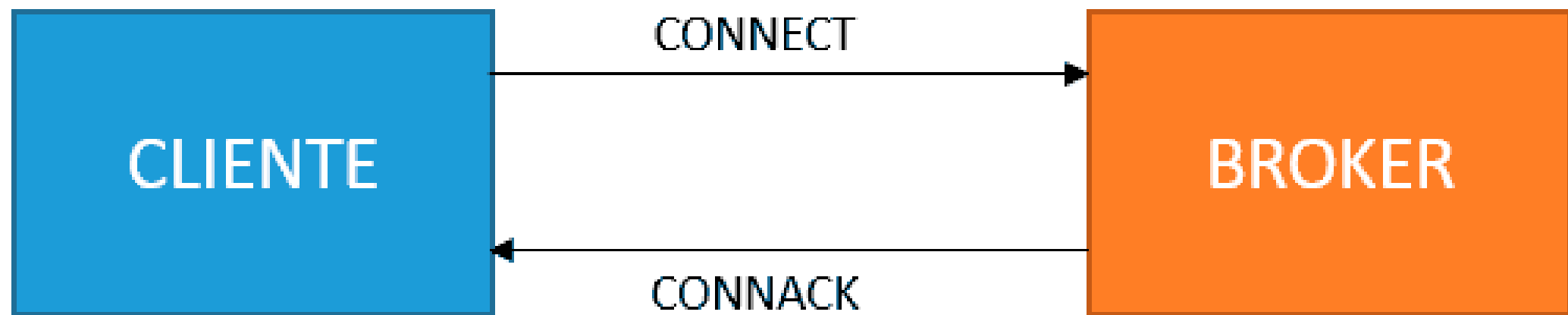


## 2. ¿Cómo funciona MQTT?

Los clientes inician una conexión TCP/IP con el broker, el cual mantiene un registro de los clientes conectados. Esta conexión se mantiene abierta hasta que el cliente la finaliza. Por defecto, MQTT emplea el puerto 1883 y el 8883 cuando funciona sobre TLS.

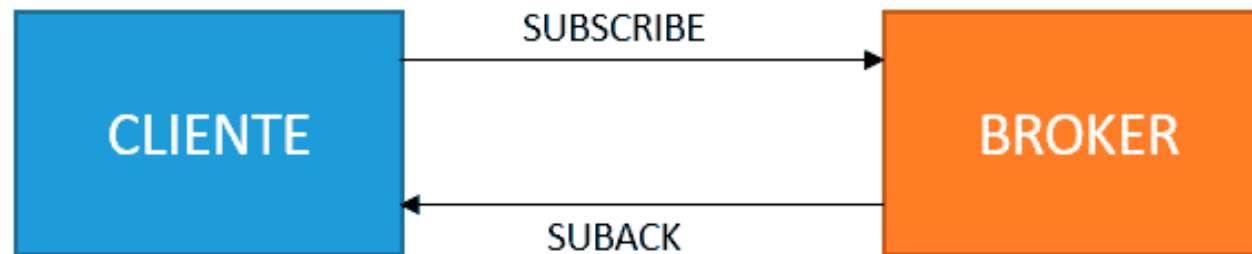
Para ello el cliente envía un mensaje CONNECT que contiene información necesaria (nombre de usuario, contraseña, client-id...).

El broker responde con un mensaje CONNACK, que contiene el resultado de la conexión (aceptada, rechazada, etc).



## 2. ¿Cómo funciona MQTT?

Para enviar los mensajes el cliente emplea mensajes PUBLISH, que contienen el topic y el payload.



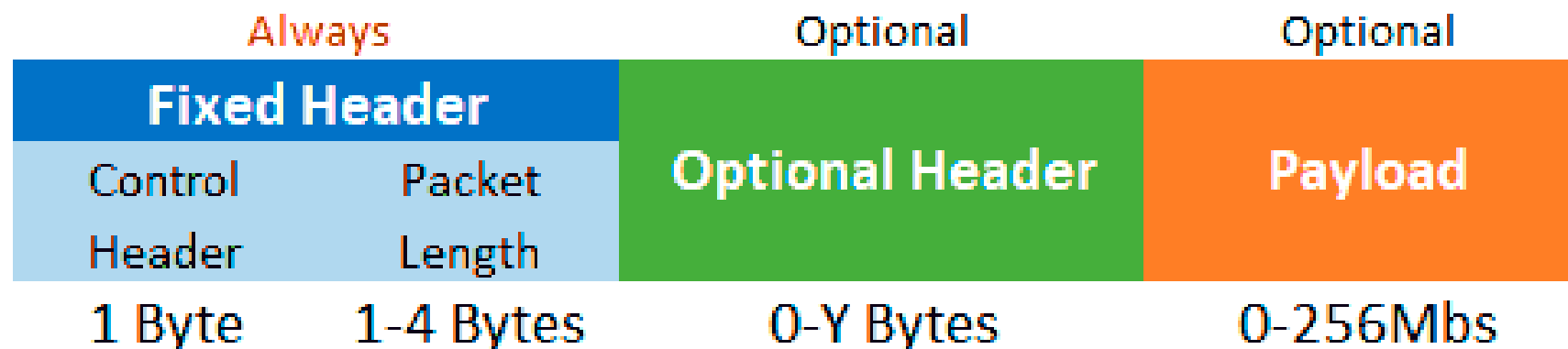
Para suscribirse y desuscribirse se emplean mensajes SUBSCRIBE y UNSUBSCRIBE, que el servidor responde con SUBACK y UNSUBACK.



Por otro lado, para asegurar que la conexión está activa los clientes mandan periódicamente un mensaje **PINGREQ** que es respondido por el servidor con un PINGRESP. Finalmente, el cliente se desconecta enviando un mensaje de DISCONNECT.

### 3. Estructura de un mensaje MQTT.

Uno de los componentes más importantes del protocolo MQTT es **la definición y tipología de los mensajes**, ya que son una de las bases de la agilidad en la que radica su fortaleza. Cada mensaje consta de 3 partes:



- **Cabecera fija.** Ocupa 2 a 5 bytes, obligatorio. Consta de un código de control, que identifica el tipo de mensaje enviado, y de la longitud del mensaje. La longitud se codifica en 1 a 4 bytes, de los cuales se emplean los 7 primeros bits, y el último es un bit de continuidad.
- **Cabecera variable.** Opcional, contiene información adicional que es necesaria en ciertos mensajes o situaciones.
- **Contenido(payload).** Es el contenido real del mensaje. Puede tener un máximo de 256 Mb aunque en implementaciones reales el máximo es de 2 a 4 kB.

## 4. MENSAJES Y TOPICS

En MQTT, cuando publicas un mensaje en un tópico, el broker lo entrega a **todos los clientes que estén suscritos a ese tópico o a un patrón de tópicos que coincida**.

### - Entrega a Todos los Suscriptores:

Cada vez que se publica un mensaje, todos los clientes con una suscripción que coincida con el tópico recibirán el mensaje. No existe un mecanismo de "exclusión" o distribución selectiva como en fanout o topic en RabbitMQ.

### - Uso de Comodines:

MQTT permite el uso de comodines para suscripciones:

- **‘+’** (más): Representa un solo nivel en la jerarquía del tópico.
  - Ejemplo: Suscribirte a **iabd2425/+/led** capturará **iabd2425/esp32/led** o **iabd2425/sensor/led**.
- **‘#’** (almohadilla): Representa múltiples niveles y debe usarse al final del patrón.
  - Ejemplo: Suscribirte a **iabd2425/#** capturará cualquier tópico que empiece por **iabd2425/**.

Estos comodines permiten cierta flexibilidad similar a los patrones en RabbitMQ, pero la forma de entrega siempre es la misma: cada mensaje se reenvía a todos los clientes cuyas suscripciones coincidan.



# 4. Calidad de Servicio.

MQTT dispone de un **mecanismo de calidad del servicio o QoS**, entendido como la forma de gestionar la robustez del envío de mensajes al cliente ante fallos (por ejemplo, de conectividad).

MQTT tiene tres niveles QoS posibles.

- **QoS 0 unacknowledged (at most one)**: El mensaje se envía una única vez sin requerir ningún acuse de recibo. No hay confirmación de que el mensaje haya sido recibido; el publicador lo envía y se asume que, si se pierde en el camino, no se reenvía.

- **QoS 1 acknowledged (at least one)**: El mensaje se envía y se reenvía hasta recibir una confirmación (PUBACK) del broker, lo que garantiza que el mensaje llegue al menos una vez. El publicador envía el mensaje y espera una respuesta de acuse de recibo (PUBACK). Si no se recibe el acuse en un tiempo determinado, se vuelve a enviar el mensaje. Puede duplicar mensajes.

- **QoS 2 assured (exactly one)**. Es el nivel más alto de confiabilidad, ya que garantiza que cada mensaje se entregue al suscriptor exactamente una vez, sin duplicados. Utiliza un protocolo de cuatro pasos (handshake) que involucra múltiples intercambios (PUB, PUBREC, PUBREL y PUBCOMP). El publicador envía el mensaje (PUBLISH). El broker responde con un PUBREC (recibo). El publicador envía un PUBREL (solicitud de liberación). El broker responde finalmente con un PUBCOMP (confirmación completa).

El nivel de QoS se configura en el cliente, tanto en el publicador como en el suscriptor, mediante los parámetros que se pasan en las funciones de publicación o suscripción.

Usar un nivel u otro **depende de las características y necesidades de fiabilidad de nuestro sistema**. Lógicamente, un nivel de QoS superior requiere un mayor intercambio mayor de mensajes de verificación con el cliente y, por tanto, mayor carga al sistema.



# 5. RETENCION.

La retención en MQTT se refiere a la capacidad del broker para almacenar el último mensaje enviado en un tópico que tenga activado el flag de retención. Esto significa que:

- **Mensaje retenido:** Cuando un publicador envía un mensaje con el flag de retención activado, el broker lo guarda.
- **Nuevos suscriptores:** Si después un cliente se suscribe a ese tópico, el broker le envía automáticamente el último mensaje retenido, permitiéndole conocer el estado actual o la última información publicada.
- **Aplicación:** Es especialmente útil para enviar el estado inicial de dispositivos o servicios sin esperar a que se publique un nuevo mensaje.

## 5. Seguridad en MQTT.

- La seguridad siempre debe ser un factor importante a considerar en cualquier sistema de comunicación M2M. El protocolo MQTT **dispone de distintas medidas de seguridad** que podemos adoptar para proteger las comunicaciones.
- Esto incluye **transporte SSL/TLS y autenticación por usuario y contraseña o mediante certificado**. Sin embargo, hay que tener en cuenta que muchos de los dispositivos IoT disponen de escasa capacidad, por lo que el SSL/TLS puede suponer una carga de proceso importante.
- En muchos casos, la autenticación consiste en una contraseña y usuario que son enviados como texto plano. Por último, también es posible configurar el broker para aceptar conexiones anónimas.
- Todo esto debe ser tenido en cuenta a la hora de configurar un sistema MQTT, y entender los riesgos de cada uno de ellos, así como su impacto en la eficiencia del sistema.

## 6. Ventajas de MQTT.

Son varias las ventajas del protocolo MQTT como sistema de comunicación M2M. Por un lado, **tenemos todas las ventajas del patrón pub/sub** que vimos en la entrada anterior, como son escalabilidad, asincronismo, desacoplamiento entre clientes.

Además, MQTT aporta una serie de características que le han hecho sobre salir sobre otros competidores. La principal, como hemos mencionado, **es su sencillez y ligereza**. Esto lo hace adecuado para aplicaciones IoT, donde frecuentemente se emplean **dispositivos de escasa potencia**.

Además, esta menor necesidad de recursos se traduce en **un menor consumo de energía**, lo cual es interesante en dispositivos que funcionan 24/7 y muy especialmente en dispositivos alimentados por batería.

Otra consecuencia de la ligereza del protocolo MQTT es que **requiere un ancho de banda mínimo**, lo cual es importante en redes inalámbricas, o conexiones con posibles problemas de calidad.

Por último, MQTT dispone de medidas adicionales importantes, como la **seguridad y calidad del servicio** (QoS). Por último, es una solución largamente testada y consolidada, que **aporta robustez y fiabilidad**.